

Assignment-2 Report

Basic Structure of code:

- The datasets contained edgelists. I repurposed the code for reading data in Assignment 1.
- However, since the total number of edges and the total number of vertices were not provided, I had to do a two-pass reading.
- One pass calculated the number of nodes and edges only.
- The second one read the actual edges.
- I did this so I could use vectors instead of maps (as I did maps, which affected overall time in Assignment 1).
- The time calculation was repurposed from Assignment 1.
- The Edgelist and the DegreeList were global to avoid unnecessary passing overhead.

Understanding the algorithm and the problem:

- “Forward algorithm” was used. We had to calculate the number of triangles in a graph.
- Since each triangle could be counted three times, I made a degree array, which only counted once based on the condition:

```
if ((deg[v] < deg[u]) || (deg[v] == deg[u] && v < u)) continue;
```

And for the third vertex

```
if ((deg[w] > deg[u] || (deg[w] == deg[u] && w > u)) &&  
    (deg[w] > deg[v] || (deg[w] == deg[v] && w > v))) ...
```

- This basically kept a specific order of the degrees and used the IDs of the vertices as a fallback in case of equal degrees.
- The condition is basically:

If the degree of $v >$ degree of u , then proceed to find the third vertex. And the degree of the third vertex must be greater than both; only then will the triangle counter be incremented.

- If these conditions are not met, then either or both respective counters for u and v are incremented.
- This algorithm has three loops; Outer for, inner for, and inner-most while.
- Inner-most while loops cannot be parallelized for obvious reasons.
- Inner for is dependent on outer for, so only the outer for can be parallelized.
- The sequence was the implementation of the aforementioned algorithm.
- The edgelist was sorted before calling compute, and duplicates were removed.

Part-A: SIMD intrinsics:

- A separate function was created and called instead of the while loop block.
- Here, we had two while loops, one for intrinsics and the other for regular (fallback).
- The intrinsic while loop kept going till both u and v integer lines had at least 8 integers left; if not, then the fallback loop processed them normally.
- First, the data from Edgelist[u] was loaded.
- If the greatest element of Edgelist[u] was less than the least element of Edgelist[v] and vice versa, then the computation was skipped. This was done in O(1) time since the vectors were sorted.
- Next, there was a for loop, in which elements in Edgelist[v] (8 elements) were being broadcast.
- The integer line from Edgelist[u] and the broadcast values were compared, and -1 was stored in the place of the same integer in the mask.
- The location of -1 (location of matching value) was calculated from the mask via:

```
int matched_idx = __builtin_ctz(mask) / 4;
```

And the third vertex became:

```
int w = graph_global[you][i + matched_idx];
```

- The rest of the computation was basically the same.

Part-B: OpenMP simd:

- Since we had to apply OpenMP SIMD, which can only be applied on a for loop, we had to rewrite the while loop.
- The while loop was replaced with two inner for loops with a very simple matching algorithm.
- The header used was

```
#pragma omp simd reduction(+:local_count)
```

- “Reduction” is here to prevent race conditions in local_count, and simd is for combining lanes of integers.
- Reduction is just managing the lines of integers. Only one thread is being used here.
- This will cause a slowdown because almost no parallelization is done and there are four back-to-back inner loops.

For all further implementations, the number of threads used is 4.

Part-C: OpenMP:

- This had the same implementation as in the Sequential part, with this header at the beginning of the first for loop:

```
#pragma omp parallel for schedule(dynamic)  
reduction(+:total_triangles)
```

- “Parallel for” ensured work sharing was done, dynamic scheduling allowed the faster threads to do more work (good load balancing, some scheduling overhead), and “reduction” prevented race conditions and helped with synchronization.

Part-D: OpenMP + SIMD intrinsics:

- This had the same implementation as in part A, with the addition of the header used in part C.

Profiling and Results:

- Three datasets were used from the provided resources. All answers were crosschecked across the four implementations.
- The profiling of all three data sets along with graphs have been done in separate pdfs.